

GIORNO 03

Input / Output: un programma che chiede, ascolta e risponde

Indice della giornata

1. Ripasso e domande
2. Un programma che parla e ascolta
3. `input()`: come funziona davvero
4. Scrivere un prompt che si capisce
5. L'igiene del dato in ingresso
6. Convertire ciò che arriva
7. Più valori in una sola riga
8. L'output come prodotto finito
9. Costruire tabelle a larghezza fissa
10. `print()` avanzato
11. Maschere di inserimento
12. Il ciclo di vita di un dato
13. Tre casi reali a confronto
14. Quello che oggi non sappiamo ancora fare

Cosa saprete fare stasera

Leggere dallo standard input, normalizzare il dato, convertirlo nel tipo giusto.

Spiegare perché `input()` restituisce sempre un valore di tipo `str`.

Ricavare i campi di un record da una riga sola, con `.split()` e l'unpacking.

Comporre un output a colonne con gli specificatori di formato delle f-string.

Elencare cinque input che fanno saltare il programma, e nominare l'eccezione.

Il pezzo scomodo: da oggi nel programma entra qualcosa che non avete scritto voi, quindi può entrare qualunque cosa.

E il programma di oggi, davanti a qualunque cosa, si schianta.

1. RIPASSO E DOMANDE

I tre scogli di ieri, e cosa cambia da stamattina

I tre scogli di ieri

Scoglio di ieri	Il rimedio	Oggi serve per
"5" + 3 non si può fare	Convertire con int() o float()	Ogni numero che arriva dalla tastiera
nome.upper() non cambia nome	Riassegnare: nome = nome.upper()	Ripulire il dato in ingresso
print("Totale: " + totale)	f-string: print(f"Totale: {totale}")	Costruire l'output allineato

La riga da correggere insieme

```
prezzo = "49.90"  
quantita = 3  
print(prezzo * quantita)
```

```
# a schermo:  
49.9049.9049.90
```

Non solleva nessuna eccezione, ed è peggio: gli operandi sono di tipo diverso e per Python testo per intero vuol dire ripeti.

Cosa cambia da stamattina

Fino a ieri	Da oggi
<code>prezzo = 49.90</code> scritto nel file	<code>prezzo = float(input("Prezzo: "))</code>
Il programma fa sempre la stessa cosa	Fa cose diverse a ogni lancio
Se il dato è sbagliato è colpa vostra	È colpa di chi digita
Per provarlo, lo lanciate	Per provarlo, lo lanciate E digitate

2. UN PROGRAMMA CHE PARLA E ASCOLTA

Ingresso, elaborazione, uscita

Lo schema che non cambierà mai più

INGRESSO (input)	->	ELABORAZIONE (i calcoli)	->	USCITA (output)
Bancomat	tessera e PIN	-> controlla il saldo	->	soldi e ricevuta
Bilancia	peso e tasto	-> moltiplica	->	etichetta stampata
Parcheggio	biglietto	-> calcola le ore	->	sbarra aperta

Lo schema ha un nome, ed è quello che troverete sui documenti di analisi: modello ingresso-elaborazione-uscita. Fino a ieri avete scritto solo la parte destra; oggi montiamo quella sinistra.

I due canali: una bocca e un orecchio

Canale	Nome tecnico	Come si usa	Dove finisce / da dove arriva
Uscita	standard output (stdout)	print(...)	La finestra del terminale
Ingresso	standard input (stdin)	input(...)	La tastiera, fino all'invio

print() scrive i suoi argomenti sullo standard output e finisce lì: nessun file viene toccato. La finestra dove i due canali sbucano si chiama terminale, o console. I file arrivano più avanti nel corso.

Il primo programma che dialoga

```
nome = input("Come ti chiami? ")  
print("Buongiorno,", nome)  
print("Benvenuto in NovaStore.")
```

--- a schermo ---

```
Come ti chiami? Giulia  
Buongiorno, Giulia  
Benvenuto in NovaStore.
```

Metà di quella prima riga la scrive il programma, metà la digitate voi: il prompt viaggia sullo standard output, la risposta arriva dallo standard input. Lo spazio fra i due argomenti di print() lo mette lui, per via di un parametro che si chiama sep.

L'ordine di esecuzione, e il NameError che ne segue

```
print("Passo 1: preparo il modulo")

risposta = input("Passo 2: dimmi il tuo nome -> ")

print("Passo 3: modulo compilato per", risposta)

# fra il passo 1 e il passo 3 l'esecuzione resta sospesa su quella riga:
# input() non ha timeout, e nessun valore di ripiego scatta al posto vostro

# e se il saluto sta SOPRA la domanda:

print("Buongiorno,", nome)

nome = input("Come ti chiami? ")
```

NameError: name 'nome' is not defined

risposta = input(...) è una istruzione; dentro, input(...) è una espressione, cioè qualcosa che viene valutato e produce un valore. NameError è la prima eccezione della giornata e si traduce «questo nome non lo conosco»: la riga che crea la variabile è sotto invece che sopra.

3. input(): COME FUNZIONA DAVVERO

Il pulsante del citofono

Analogia: il citofono

Premete il pulsante e restate fermi con l'orecchio attaccato.

Non aprite la porta, non salite le scale. Aspettate.

Quando qualcuno risponde, riprendete da dove eravate.

`input()` è progettato per bloccarsi.

Chi dice "mi si è bloccato il computer" ha capito perfettamente.

Fra la riga prima e la riga dopo possono passare due secondi o due ore.

L'esecuzione resta sospesa su quella riga finché non arriva l'invio:

`input()` non ha timeout, e nessun valore di ripiego scatta al posto vostro.

La forma minima

```
risposta = input("Scrivi qualcosa: ")  
print("Hai scritto:", risposta)
```

a schermo:

Scrivi qualcosa: buongiorno

Hai scritto: buongiorno

input è una funzione, le parentesi sono la chiamata, la stringa dentro è un argomento. Il prompt finisce con uno spazio: senza, la risposta si attacca alla domanda.

I tre pezzi da riconoscere

Pezzo	Che cos'è
<code>input(...)</code>	La funzione che sospende l'esecuzione e legge dallo standard input
<code>"Scrivi qualcosa: "</code>	Il prompt: l'argomento che <code>input()</code> stampa prima di mettersi in ascolto
<code>risposta =</code>	Dove finisce il valore di ritorno, cioè il valore che <code>input()</code> restituisce

Senza l'assegnazione il valore di ritorno viene prodotto e buttato via, e nessuna eccezione ve lo segnala. Per Python il prompt è facoltativo; nel corso non si scrive mai un `input()` senza prompt.

Restituisce **SEMPRE** una stringa

```
risposta = input("Quanti anni hai? ")  
print(risposta)  
print(type(risposta))
```

--- a schermo ---

```
Quanti anni hai? 42  
42  
<class 'str'>
```

Avete digitato cifre, input() ha restituito un oggetto di tipo str. Sempre: legge dallo standard input i caratteri fino all'invio, esclude il carattere di fine riga e restituisce testo.

Non tutto ciò che è fatto di cifre è un numero

```
progressivo = "0042"          # digitato dall'operatore LogiSud
```

```
print(progressivo, len(progressivo))
```

```
print(int(progressivo))
```

```
print("LS-2026-MI-" + progressivo)
```

```
# a schermo:
```

```
0042 4
```

```
42
```

```
LS-2026-MI-0042
```

int() legge quella sequenza di cifre come un letterale numerico in base 10, e gli zeri non significativi non fanno parte del valore: il numero quarantadue non ricorda come era scritto. Da qui la regola: si converte solo ciò su cui si fanno i conti. Un progressivo, un CAP, un numero di telefono restano stringhe.

Se ve ne dimenticate: TypeError

```
eta = input("Quanti anni hai? ")
print("Fra dieci anni ne avrai", eta + 10)

--- a schermo ---

Quanti anni hai? 42
Traceback (most recent call last):
  File "eta.py", line 2, in <module>
    print("Fra dieci anni ne avrai", eta + 10)
                                ~~~~^~~~
TypeError: can only concatenate str (not "int") to str
```

TypeError è un'eccezione, non un generico messaggio d'errore, e il traceback dice a quale riga. Tradotto: avete chiesto la concatenazione fra due operandi di tipo diverso. Manca la conversione esplicita.

L'invio a vuoto: l'errore che non dà errore

```
nome = input("Nome: ")
print "[" + nome + "]", len(nome)

--- a schermo, premendo solo INVIO ---

Nome:
[] 0
```

La stringa vuota non è «niente»: è un valore regolare di tipo str, lungo zero, che si assegna, si stampa e si ripulisce senza sollevare niente. Per questo il programma prosegue tranquillo. Davanti a int("") o float(""), invece, arriva ValueError.

4. SCRIVERE UN PROMPT CHE SI CAPISCE

Il modulo allo sportello: se il dato è inutilizzabile, la domanda era scritta male

I quattro difetti classici

Prompt	Cosa non va	Come si scrive
<code>input("Peso: ")</code>	In che unità? Grammi, chili?	<code>input("Peso in kg: ")</code>
<code>input("Data: ")</code>	Quale formato?	<code>input("Data (gg/mm/aaaa): ")</code>
<code>input("Importo: ")</code>	Virgola o punto?	<code>input("Importo in euro (es. 12.50): ")</code>
<code>input("Zona: ")</code>	Quali zone esistono?	<code>input("Zona (NORD/CENTRO/SUD/ISOLE): ")</code>

Nel prompt ci vanno unità di misura, formato atteso e, se aiuta, un esempio: quell'argomento è l'unica specifica che l'utente leggerà mai. Costa una riga e vi risparmia un'eccezione.

Comune di Villanova: prima versione

```
a = input("Nome: ")
b = input("CF: ")
c = input("ISEE: ")
d = input("Figli: ")
print(a, b, c, d)
```

--- a schermo ---

```
Nome: laura greco
CF: grclra85m41f839t
ISEE: 12mila
Figli: due
laura greco grclra85m41f839t 12mila due
```

Nessuna eccezione: il programma gira. Due campi numerici sono stati raccolti a parole e restano stringhe inutilizzabili. Guardate anche i nomi: a, b, c, d non dicono niente nemmeno a voi che li avete scritti.

Seconda versione: cambiano solo le domande

```
nome = input("Nome e cognome: ")
codice_fiscale = input("Codice fiscale (16 caratteri): ")
isee = input("ISEE in euro, solo cifre (es. 12000): ")
figli = input("Numero di figli a carico (solo cifre): ")
print(nome, codice_fiscale, isee, figli)
```

--- a schermo ---

```
ISEE in euro, solo cifre (es. 12000): 12000
Numero di figli a carico (solo cifre): 2
laura greco grclra85m41f839t 12000 2
```

Non è cambiata una riga di logica: sono cambiati gli argomenti passati a input(). Stessi controlli, cioè nessuno, e nonostante questo i due campi numerici adesso si possono convertire.

Istruzioni lunghe prima, prompt corto dopo

```
print("PESO DELLA SPEDIZIONE")

print("Espresso in chilogrammi. Usate il punto: 12.5, non 12,5.")

peso = input("Peso in kg: ")

# e il prompt su piu' righe fa un menu, con gli escape del primo giorno
scelta = input("Scegli il reparto:\n 1) Accettazione\n 2) Prelievi\n> ")

print("Hai scelto il reparto numero", scelta)

# a schermo:

Scegli il reparto:

1) Accettazione

2) Prelievi
```

Il prompt ≥ 2 è un argomento come un altro, e dentro valgono tutte le sequenze di escape del Giorno 01, l'a-capo compreso: viaggia sullo standard output, quindi si comporta come una stampa. La maschera è già quella definitiva; oggi il programma non sa ancora fare niente di diverso a seconda della scelta.

5. L'IGIENE DEL DATO IN INGRESSO

La ricopiatura in bella

Analogia: il modulo ricopiato in bella

Allo sportello il modulo arriva con la grafia storta,
le maiuscole a caso, un numero mezzo fuori dal riquadro.

Prima di archivarlo, qualcuno lo ricopia in bella.

Nessuno si accorge di quel lavoro.

Senza quel lavoro l'archivio è inutilizzabile.

Il nome tecnico di quella ricopiatura è normalizzazione:

portare dati scritti in modi diversi a una forma unica, decisa da voi.

`.strip()`, `.lower()`, `.replace()` sono la ricopiatura in bella.

Da oggi si fanno sempre, anche quando sembra che non servano.

.strip(): gli spazi che non si vedono

```
zona_grezza = input("Zona: ")
zona = zona_grezza.strip()

print "[" + zona_grezza + "]", len(zona_grezza)
print "[" + zona + "]", len(zona)
```

--- a schermo, digitando due spazi e poi SUD ---

```
Zona:   SUD
[  SUD] 5
[SUD] 3
```

Cinque caratteri contro tre. Senza argomenti .strip() non toglie «gli spazi»: toglie i caratteri di spaziatura, cioè lo spazio, la tabulazione e il carattere di fine riga. Toglie ai lati, mai in mezzo, e restituisce una stringa nuova: per questo zona_grezza è ancora lì con i suoi due spazi.

Uniformare: le quattro pulizie

Metodo	' mario rossi ' diventa	Quando si usa
<code>.strip().upper()</code>	'MARIO ROSSI'	Codici, sigle, zone, codice fiscale
<code>.strip().lower()</code>	'mario rossi'	Email, risposte, confronti
<code>.strip().title()</code>	'Mario Rossi'	Nomi e cognomi da stampare
<code>.strip().capitalize()</code>	'Mario rossi'	Frase libere: motivi, note, causali

.title() mette la maiuscola a ogni parola e abbassa tutto il resto: 'comune di villanova' -> 'Comune Di Villanova'. Per lui una parola finisce dove finiscono le lettere, apostrofo compreso: da qui quell'All'Anca che nessuno scriverebbe.

Pulizia in pratica: nome, email, IBAN

```
nome = input("Nome: ").strip().title()
email = input("Email: ").strip().lower()
iban = input("IBAN: ").strip().replace(" ", "").upper()

print(nome)
print(email)
print(iban, len(iban))
```

--- a schermo ---

```
Nome:   mario rossi
Email:   Mario.Rossi@Posta.IT
IBAN: it60 x054 2811 1010 0000 0123 456
Mario Rossi
mario.rossi@posta.it
IT60X0542811101000000123456 27
```

Si chiama concatenazione di chiamate e funziona per un motivo solo: ogni metodo di stringa restituisce una stringa, quindi sul risultato se ne può chiamare un altro. Si valuta da sinistra a destra, come una catena di montaggio. Regola del corso: massimo tre.

L'ordine della catena non è indifferente

```
print("nord centro".title().replace(" ", ""))  
print("nord centro".replace(" ", "").title())
```

NordCentro

Nordcentro

```
# e l'immutabilita', che e' il motivo per cui il grezzo sopravvive:  
grezzo = "  SUD  "  
pulito = grezzo.strip()  
  
print "[" + grezzo + "]", len(grezzo)  
print "[" + pulito + "]", len(pulito)
```

Nella prima riga `.title()` vede due parole, nella seconda lo spazio è già sparito e ne vede una: quando una catena non dà quello che vi aspettate, provate a invertirne due pezzi. Sotto: in Python una stringa è immutabile, nessun metodo la modifica sul posto. Restituiscono una stringa nuova, e l'originale resta per il registro.

La virgola italiana: il crash e la difesa

```
prezzo = float(input("Prezzo in euro: "))
```

```
--- digitando 12,50 ---
```

```
ValueError: could not convert string to float: '12,50'
```

```
--- la difesa e' una riga ---
```

```
prezzo_grezzo = input("Prezzo in euro: ")  
prezzo = float(prezzo_grezzo.strip().replace(",", ".", "."))  
print(f"Registrato: {prezzo:.2f} euro")
```

```
Prezzo in euro: 12,50  
Registrato: 12.50 euro
```

Su una stringa senza virgola .replace() restituisce la stessa stringa e non solleva niente: la potete mettere sempre, senza sapere come scriverà chi digita. Il suo terzo argomento, che limita quante sostituzioni fare, ha come valore predefinito «tutte quante».

La tabella dell'igiene

Tipo di dato	Pulizia da applicare
Nome, cognome	<code>.strip().title()</code>
Email	<code>.strip().lower()</code>
Codice fiscale, sigla, zona	<code>.strip().upper()</code>
Importo, peso, misura	<code>.strip().replace(",", ".")</code> poi <code>float()</code>
Quantità, età, anno	<code>.strip()</code> poi <code>int()</code>
Telefono	<code>.strip().replace(" ", "").replace("-", "")</code>
IBAN, codice a barre	<code>.strip().replace(" ", "").upper()</code>
Frase libera (causale, motivo)	<code>.strip()</code> e basta

Una riga per tipo di dato: è il vostro prontuario di normalizzazione. Regola del corso: massimo tre metodi in catena su una riga; se ne servono cinque, o se si superano gli 88 caratteri, si spezza in due assegnazioni.

6. CONVERTIRE CIÒ CHE ARRIVA

`int()`, `float()` e il primo `ValueError` della giornata

Le due conversioni che userete sempre

```
eta_testo = input("Eta: ").strip()
```

```
eta = int(eta_testo)
```

```
prezzo_testo = input("Prezzo in euro: ").strip()
```

```
prezzo = float(prezzo_testo)
```

```
print(eta + 1)
```

```
print(prezzo * 2)
```

```
# a schermo:
```

```
Eta: 41
```

```
Prezzo in euro: 49.90
```

⁴²
Queste due righe sono una conversione esplicita di tipo, in gergo casting: la chiedete voi, scrivendo `int(...)` o `float(...)` attorno al valore. Fra testo e numeri Python non fa mai conversione implicita. Due righe e non `int(input(...))`: la stringa originale serve ancora, e presto vorrete controllarla.

Il momento preciso in cui il programma esplode

```
eta_testo = input("Eta: ").strip()
eta = int(eta_testo)
print("Fra dieci anni ne avrai", eta + 10)
```

--- a schermo, digitando venti ---

Eta: venti

Traceback (most recent call last):

File "eta.py", line 2, in <module>

eta = int(eta_testo)

^^^^^^^^^^^^^^^^

ValueError: invalid literal for int() with base 10: 'venti'

int(testo) riesce se il testo, tolti gli spazi ai lati, è un letterale intero in base 10: un segno facoltativo e poi solo cifre. Tutto il resto solleva ValueError. Il print della riga 3 non è mai stato eseguito: l'eccezione ha fermato lì il programma.

Leggere il traceback del ValueError

Pezzo	Cosa dice
ValueError	Il tipo era giusto (una stringa), ma il valore no
invalid literal for int()	Non riesco a leggere questo come numero intero
with base 10	In base dieci, cioè quella normale
'venti'	Il valore esatto che ha fatto saltare tutto, fra apici

Il traceback si legge dal basso: prima il nome dell'eccezione, poi la riga che l'ha sollevata. Quel valore fra apici è oro: nel registro di un programma vero è spesso l'unica cosa che avete.

Conversioni che riescono e conversioni che no

Espressione	Risultato
<code>int("42")</code>	42
<code>int(" 42 ")</code>	42 — gli spazi ai lati <code>int()</code> li tollera
<code>int("42 anni")</code>	ValueError
<code>int("")</code>	ValueError
<code>int("3.5")</code>	ValueError — con il punto no
<code>int("-7")</code>	-7 — il segno meno sì
<code>float("3.5")</code>	3.5
<code>float("3,5")</code>	ValueError — la virgola no
<code>float("42")</code>	42.0 — un intero è un float valido
<code>float("")</code>	ValueError

`int("3.5")` solleva `ValueError` e sorprende tutti: `int()` pretende un letterale intero, non arrotonda e non tronca il testo. La strada tecnica è `int(float("3.5"))`, che tronca verso lo zero; la strada giusta, quasi sempre, è un prompt migliore.

Dimenticarsi di convertire: il crash e il caso muto

```
quantita = input("Quantita: ")  
prezzo = 49.90  
print("Totale:", prezzo * quantita)
```

```
Quantita: 3  
TypeError: can't multiply sequence by non-int of type 'float'
```

--- e il fratello cattivo, quello che NON da' errore ---

```
SCATOLE = 4  
quantita = input("Pezzi per scatola: ")  
print("Pezzi totali:", quantita * SCATOLE)
```

```
Pezzi per scatola: 3  
Pezzi totali: 3333
```

La «sequenza» del primo messaggio è la vostra stringa. Sotto, gli stessi due operandi in ordine diverso: testo per intero, per Python, vuol dire ripeti. Quattro volte il carattere 3, nessuna eccezione, nessun avviso.

La bilancia del reparto ortofrutta

Se appoggiate la mano sul piatto insieme alle mele, la bilancia non protesta: stampa un'etichetta con il peso sbagliato.

Spazzatura dentro, spazzatura fuori.

Un'età di -5 passa. Un prezzo di 999999 passa. Un peso di 0 passa.

"Ma non si può dire al programma di non morire?"

Sì: esiste uno strumento per raccogliere l'eccezione dopo che è stata sollevata, invece di prevenirla. Si chiama gestione delle eccezioni, try / except, e non è materia di queste undici giornate.

Qui ci fermiamo a prevenire, ed è la prima linea di difesa.

Lo strumento per dire di no arriva nella prossima giornata.

7. PIÙ VALORI IN UNA SOLA RIGA

LogiSud: ottanta spedizioni al giorno, e duecentoquaranta invii

.split(): tagliare su un separatore

```
riga = "SUD;12.5;urgente"
```

```
pezzi = riga.split(";")
```

```
print(pezzi)
```

```
# a schermo:
```

```
['SUD', '12.5', 'urgente']
```

Da qui in avanti ai pezzi diamo il loro nome: la riga è un record, cioè tutti i dati di una sola spedizione; ogni pezzo è un campo; il punto e virgola è il separatore, o delimitatore. Le parentesi quadre sono una lista: la studierete con le strutture dati, oggi ci basta aprirla.

Il conto dei campi: i delimitatori più uno

```
# con un separatore esplicito i campi sono sempre i delimitatori piu' uno,  
# anche quando un campo e' vuoto  
print("SUD;;urgente".split(";"))  
print("SUD".split(";"))
```

```
['SUD', '', 'urgente']  
['SUD']
```

```
# e sui casi estremi le due forme si comportano in modo opposto:  
print("".split())  
print("".split(";"))
```

```
[]  
['']
```

L'unpacking: tre nomi in un colpo solo

```
riga = "SUD;12.5;urgente"  
zona, peso_testo, urgenza = riga.split(";")
```

```
print(zona)  
print(peso_testo)  
print(urgenza)
```

```
SUD  
12.5  
urgente
```

--- funziona anche con due ---

```
riga = "Ferrante;NORD"  
cognome, zona = riga.split(";")  
print(cognome, "opera nella zona", zona)
```

```
Ferrante opera nella zona NORD
```

Il nome per esteso è unpacking, in italiano spaccettamento, ed è una assegnazione multipla: un solo uguale, più nomi a sinistra. Devono essere esattamente quanti i campi a destra; se i due numeri non coincidono solleva ValueError e non assegna niente.

Quando i pezzi sono di numero sbagliato

```
zona, peso_testo, urgenza = "SUD;12.5".split(";")
```

```
ValueError: not enough values to unpack (expected 3, got 2)
```

```
zona, peso_testo, urgenza = "SUD;12.5;urgente;fragile".split(";")
```

```
ValueError: too many values to unpack (expected 3)
```

```
zona, peso_testo, urgenza = "SUD,12.5,urgente".split(";")
```

```
ValueError: not enough values to unpack (expected 3, got 1)
```

got 1: non ha trovato nemmeno un punto e virgola. Il colpevole è il separatore.

Pulire dopo lo split, non prima

```
riga = "sud; 12,5 ;URGENTE"
zona_grezza, peso_grezzo, urgenza_grezza = riga.split(";")
```

```
print "[" + zona_grezza + "]"
print "[" + peso_grezzo + "]"
print "[" + urgenza_grezza + "]"
```

```
[sud]
[ 12,5 ]
[URGENTE]
```

--- quindi si pulisce dopo, pezzo per pezzo ---

```
zona = zona_grezza.strip().upper()
peso_kg = float(peso_grezzo.strip().replace(",", ".", ""))
urgenza = urgenza_grezza.strip().lower()
print(f"{zona} | {peso_kg:.2f} kg | {urgenza}")
```

```
SUD | 12.50 kg | urgente
```

.split() non normalizza niente: gli spazi attorno ai campi restano dentro i campi. Quindi ogni campo si ripulisce per conto suo, dopo lo split. Sei righe per una riga digitata, ed è il minimo: quella riga arriva da una persona di fretta.

Quale separatore

Separatore	Quando	Rischio
;	Dati strutturati, sempre	Basso: raro nei testi italiani
,	Mai in questo corso	Compare nei decimali e nelle frasi
.split() senza argomenti	Nome e cognome, parole	I nomi composti
	Alternativa a ;	Va spiegato all'utente

Il prompt deve dire il formato in modo inequivocabile, delimitatore compreso: 'Spedizione (zona;peso;urgenza): '. Mai 'Dati spedizione: '. La riga che conta è la seconda: basta una causale con un elenco dentro, o un importo scritto con la virgola, e i campi diventano quattro.

8. L'OUTPUT COME PRODOTTO FINITO

Il cameriere che ripete l'ordine

Chi legge il vostro output (non voi)

Domanda	Se la risposta è no
Il numero che serve si trova in due secondi?	Isolatelo, dategli una riga sua
Le sezioni sono distinguibili?	Aggiungete righe di separazione
I numeri sono incolonnati?	Allineateli a destra con le f-string
Gli importi hanno due decimali?	:.2f, sempre, sugli euro
C'è scritto di cosa si tratta?	Intestazione in cima, con il nome dell'ente

"Due margherite e un'acqua. Confermo?" Nel software quel riepilogo si chiama eco, e non è educazione: è controllo qualità. Costa cinque secondi e intercetta l'errore prima che arrivi in cucina.

Lo stesso calcolo, presentato

```
articolo = "Cuffie Bluetooth"

quantita = 2

prezzo = 49.90

totale = prezzo * quantita


print("=" * 50)

print("NOVASTORE - RIEPILOGO")

print("=" * 50)

print(f"Articolo:    {articolo}")

print(f"Quantita:     {quantita}")

print(f"Prezzo:        {prezzo:.2f} euro")

print("-" * 50)

print(f"TOTALE:         {totale:.2f} euro")

print("=" * 50)
```

Prima e dopo

--- prima: quattro print nudi ---

Cuffie Bluetooth

2

49.9

99.8

--- dopo ---

=====

NOVASTORE - RIEPILOGO

=====

Articolo: Cuffie Bluetooth

Quantita: 2

Prezzo: 49.90 euro

TOTALE: 99.80 euro

=====

Gli zeri dei centesimi tornano con :.2f. Il totale ha una riga tutta sua.

Le righe di separazione

Segno	Uso
=	Apertura e chiusura del documento, titolo
-	Separazione fra sezioni interne
#	Cornice di un riquadro
.	Riempimento fra etichetta e valore

*Si mettono in costante in cima al file: $LARGHEZZA = 50$, $RIGA_DOPPIA = '=' * 50$. Così la larghezza del documento si cambia in un punto solo, e l'operatore * su una stringa è la ripetizione del Giorno 02.*

9. COSTRUIRE TABELLE A LARGHEZZA FISSA

Colonne che non ballano

Senza allineamento, e con

```
print("Cuffie Bluetooth", 2, 99.80)
print("Hub USB-C", 1, 27.00)
print("Mouse verticale", 3, 103.50)
```

```
Cuffie Bluetooth 2 99.8
Hub USB-C 1 27.0
Mouse verticale 3 103.5
```

--- il rimedio: una larghezza fissa per ogni colonna ---

```
print(f"[{'Cuffie':<12}]")
print(f"[{'Cuffie':>12}]")
print(f"[{'Cuffie':^12}]")
```

```
[Cuffie      ]
[      Cuffie]
[   Cuffie   ]
```

Le colonne perdono l'allineamento e i decimali spariscono. Il rimedio è dare a ogni colonna una larghezza fissa: dentro una f-string ogni coppia di graffe è un campo di sostituzione, e a destra dei due punti c'è lo specificatore di formato.

Le specifiche che servono oggi

Specifica	Significato	Si usa per
:<12	A sinistra, larghezza 12	Testo: nomi, descrizioni
:>12	A destra, larghezza 12	Numeri: importi, quantità
:^12	Centrato, larghezza 12	Titoli, intestazioni brevi
:>10.2f	A destra in 10, due decimali	Gli euro, sempre
:>6.1%	A destra in 6, percentuale	Quote, sconti

Testo a sinistra, numeri a destra: così le unità finiscono sotto le unità e l'occhio confronta. Lo specificatore si legge sempre nello stesso ordine, e la slide dopo lo smonta pezzo per pezzo.

Smontare uno specificatore di formato

```
# {valore:>10.2f}

#      || | |
#      || | +-- tipo di presentazione: decimale, mai esponenziale
#      || +---- precisione: due cifre dopo il punto
#      |+----- larghezza MINIMA: un pavimento, non una gabbia
#      +----- allineamento: spinge il valore a destra
```

```
print(f"{'a destra:':<12}{9.5:>10.2f}")
print(f"{'a destra:':<12}{1234.5:>10.2f}")
```

```
a destra:          9.50
a destra:        1234.50
```

```
# senza segno, l'allineamento predefinito dipende dal tipo:
print(f"[{'Cuffie':12}]")
print(f"[{49.9:12.2f}]")
```

```
[Cuffie          ]
[          49.90]
```

La tabella completa

```
print("=" * 50)

print(f"{'ARTICOLO':<22}{'Q.TA':>4}{'PREZZO':>10}{'TOTALE':>14}")

print("-" * 50)

print(f"{'Cuffie Bluetooth':<22}{2:>4}{49.90:>10.2f}{99.80:>14.2f}")

print(f"{'Hub USB-C':<22}{1:>4}{27.00:>10.2f}{27.00:>14.2f}")

print("-" * 50)

print(f"{'TOTALE':<36}{126.80:>14.2f}")

print("=" * 50)
```

Il conto delle larghezze si fa con carta e penna, una volta sola, prima di scrivere il codice: $22 + 4 + 10 + 14 = 50$, e sulla riga del totale $36 + 14 = 50$. Se la somma non torna, la tabella balla rispetto ai separatori e si vede subito.

L'output della tabella completa

ARTICOLO	Q.TA	PREZZO	TOTALE
Cuffie Bluetooth	2	49.90	99.80
Hub USB-C	1	27.00	27.00
TOTALE			126.80

La cifra del totale cade esattamente sotto la colonna dei totali. Non è un caso.

Due trappole delle colonne

--- 1. l'ultima colonna non si allinea mai a sinistra ---

```
print(f{'Cuffie Bluetooth':<22}{2:>4}pezzi")
print(f{'Cuffie Bluetooth':<22}{2:>4} pezzi")
```

```
Cuffie Bluetooth      2pezzi
Cuffie Bluetooth      2 pezzi
```

--- 2. il testo piu' lungo della colonna sfonda ---

```
print(f{'Monitor 27 pollici curvo professionale':<22}{1:>4}")
```

```
Monitor 27 pollici curvo professionale  1
```

--- il rimedio: lo slicing di ieri ---

```
descrizione = "Monitor 27 pollici curvo professionale"
print(f{descrizione[0:22]:<22}{1:>4}")
```

```
Monitor 27 pollici cur  1
```

Un :<20 finale lascia spazi invisibili in fondo alla riga: l'ultima colonna non si allinea mai a sinistra. E nessuno specificatore taglia, quindi tagliate voi con lo slicing: nell'intervallo l'estremo sinistro è incluso e il destro escluso, ed è per questo che [0:22] e la larghezza 22 sono lo stesso numero.

10. `print()` AVANZATO

sep, end, tabulazioni, riquadri, marcatori

sep e la tabulazione, con il suo limite

```
print("SUD", 12.5, 8.50)
print("SUD", 12.5, 8.50, sep=" | ")
```

```
SUD 12.5 8.5
SUD | 12.5 | 8.5
```

--- \t salta alla prossima tacca, ogni otto caratteri ---

```
print("Zona:\tSUD")
print("Peso:\t12.5")
print("Urgenza:\turgente")
```

```
Zona:    SUD
Peso:    12.5
Urgenza:          urgente
```

sep è un parametro di print() e il suo valore predefinito è uno spazio: va fra i pezzi, mai in fondo. Le prime due righe si allineano, la terza no: la tacca è una proprietà del terminale, non della stringa che avete scritto voi.

end e i riquadri

```
print("Elaborazione in corso", end="")  
print(" ... ", end="")  
print("fatto.")
```

Elaborazione in corso ... fatto.

```
print("#" * 50)  
print(f"# {'BANCA MERIDIANA':<46} #")  
print(f"# {'Operazione registrata':<46} #")  
print("#" * 50)
```

```
#####  
#  BANCA MERIDIANA                                #  
#  Operazione registrata                          #  
#####
```

Barre e marcatori di stato

```
occupati = 7
liberi = 5

print(f"Posti occupati: {'#' * occupati}{'.' * liberi} {occupati}/12")
print("[OK]  Anagrafica raccolta")
print("[!]   Codice fiscale non verificato")
print("[--]  Firma digitale non richiesta")
```

```
Posti occupati: #####..... 7/12
[OK]  Anagrafica raccolta
[!]   Codice fiscale non verificato
[--]  Firma digitale non richiesta
```

La barra è la concatenazione di due stringhe ripetute, una per i pieni e una per i vuoti, dentro un campo di sostituzione. Niente emoji: su terminali non UTF-8 diventano illeggibili, mentre [OK] e [!] funzionano ovunque.

11. MASCHERE DI INSERIMENTO

Una sequenza di domande per chi ha fretta

Le cinque regole di una maschera

Regola	Perché
Prima i dati che l'utente ha già in mano	Il documento è sul bancone: si copia e via
Dati simili vicini	Cambiare argomento a ogni domanda stanca
Sezioni dichiarate	'Sezione 2 di 3' dice quanto manca
Un campo per domanda	Due dati in una domanda producono due errori
Riepilogo finale	L'eco del cameriere del capitolo 8

Una maschera raccoglie un record campo per campo, e i gruppi di campi sono i fogli del modulo: --- ORDINANTE ---, --- BENEFICIARIO ---, --- OPERAZIONE ---.

Maschera a sei campi: Comune di Villanova

```
LARGHEZZA = 50

print("=" * LARGHEZZA)

print("COMUNE DI VILLANOVA - SPORTELLO UNICO")

print("=" * LARGHEZZA)


nome = input("Nome: ").strip().title()
cognome = input("Cognome: ").strip().title()
codice_fiscale = input("Codice fiscale: ").strip().upper()
email = input("Email: ").strip().lower()
telefono = input("Telefono: ").strip().replace(" ", "")
pratica = input("Tipo di pratica: ").strip().upper()


print()

print("=" * LARGHEZZA)

print("RIEPILOGO DELLA RICHIESTA")

print("-" * LARGHEZZA)

print(f"{'Richiedente:':<17}{nome} {cognome}")

print(f"{'Codice fiscale:':<17}{codice_fiscale}")

print(f"{'Email:':<17}{email}")

print(f"{'Telefono:':<17}{telefono}")

print(f"{'Pratica:':<17}{pratica}")

print("=" * LARGHEZZA)
```

L'output della maschera

```
Nome: laura
Cognome: greco
Codice fiscale: grclra85m41f839t
Email: Laura.Greco@Posta.IT
Telefono: 339 123 4567
Tipo di pratica: certificato di residenza
```

```
=====
RIEPILOGO DELLA RICHIESTA
-----
```

```
Richiedente:      Laura Greco
Codice fiscale:   GRCLRA85M41F839T
Email:           laura.greco@posta.it
Telefono:         3391234567
Pratica:          CERTIFICATO DI RESIDENZA
=====
```

Digitato storto in alto, archiviato dritto in basso. Fra le due metà non c'è nessun controllo: c'è solo normalizzazione, ed è già il novanta per cento del risultato.

Oggi la maschera si ferma al riepilogo

Nei programmi veri, dopo il riepilogo si chiede conferma.

Oggi possiamo chiederla, ma non possiamo ancora usarla: per scegliere in base alla risposta serve il costrutto delle decisioni, if, che arriva nella prossima giornata.

Quindi il programma di oggi mostra i dati e chiude.
È onesto dirlo, invece di far finta.

Presto, davanti a una n , saprà tornare indietro.
O meglio: saprà dire di no.

12. IL CICLO DI VITA DI UN DATO

Cinque passi, sempre in quest'ordine

I cinque passi

#	Fase	Strumento	Dove si sbaglia
1	Richiesta	<code>input("...")</code>	Prompt senza formato né unità
2	Pulizia	<code>.strip()</code> , <code>.lower()</code> , <code>.replace()</code>	Saltarla perché 'tanto è pulito'
3	Conversione	<code>int()</code> , <code>float()</code>	Dimenticarla, o farla prima di pulire
4	Calcolo	operatori aritmetici	Usare il grezzo invece del convertito
5	Presentazione	f-string con formato	Stampare 99.80000000000001

Ogni singolo campo che entra nel programma attraversa queste cinque fasi, sempre e in quest'ordine. La formula da ripetersi ad alta voce: finché è testo si normalizza, appena diventa numero si calcola.

Lo schema in codice, per esteso

```
# 1. Richiesta
peso_grezzo = input("Peso della spedizione in kg (es. 12.5): ")

# 2. Pulizia
peso_pulito = peso_grezzo.strip().replace(",", ".")

# 3. Conversione
peso_kg = float(peso_pulito)

# 4. Calcolo
COSTO_AL_KG = 1.20
costo = peso_kg * COSTO_AL_KG

# 5. Presentazione
print(f"{'Peso:':<14}{peso_kg:>8.2f} kg")
print(f"{'Costo:':<14}{costo:>8.2f} euro")

# a schermo, digitando 12,5
Peso:          12.50 kg
Costo:         15.00 euro
```

E dopo la conversione il testo non c'è più

```
peso_kg = float("12.5")      # convertito subito: qui il testo non c'e' piu'  
print(peso_kg.strip())
```

AttributeError: 'float' object has no attribute 'strip'

```
# nell'ordine giusto lo stesso lavoro non ha appigli per rompersi:  
peso_grezzo = " 12,5 "  
peso_kg = float(peso_grezzo.strip().replace(",", ".", "."))  
print(peso_kg)
```

12.5

AttributeError è un'eccezione nuova per voi e si legge come le altre, dal basso: l'oggetto di tipo float non ha un metodo che si chiama strip. Non è un refuso: quel metodo su quel tipo non esiste. .strip(), .upper() e .replace() appartengono alle stringhe, e dopo float() la stringa non c'è più.

Un nome per ogni fase

Nome	Contenuto	Tipo
peso_grezzo	Quello che ha digitato l'utente	str
peso_pulito	Dopo strip e replace	str
peso_kg	Dopo float()	float

La terza colonna non è un ornamento: è l'informazione che decide che cosa potete scrivere alla riga dopo. Su un str si chiamano i metodi di stringa e non si fa aritmetica; su un float vale il contrario. Quando non ricordate a che punto siete, type() ve lo dice in una riga.

13. TRE CASI REALI A CONFRONTO

Comune, negozio, ambulatorio: tre domini, una sola struttura

Sportello comunale: la ricevuta

```
DIRITTI_SEGRETERIA = 16.00

MARCA_DA_BOLLO = 16.00


richiedente = input("Richiedente (nome e cognome): ").strip().title()
pratica = input("Tipo di pratica: ").strip().upper()
copie_testo = input("Numero di copie richieste: ").strip()
copie = int(copie_testo)


totale = (DIRITTI_SEGRETERIA * copie) + MARCA_DA_BOLLO


print("=" * 50)
print("COMUNE DI VILLANOVA - RICEVUTA")
print("=" * 50)
print(f"{'Richiedente:':<20}{richiedente}")
print(f"{'Pratica:':<20}{pratica}")
print(f"{'Copie:':<20}{copie}")
print("-" * 50)
print(f"{'Diritti per copia:':<36}{DIRITTI_SEGRETERIA:>14.2f}")
print(f"{'Marca da bollo:':<36}{MARCA_DA_BOLLO:>14.2f}")
print(f"{'TOTALE DA VERSARE:':<36}{totale:>14.2f}")
print("=" * 50)
```

La ricevuta a schermo

Richiedente (nome e cognome): laura greco

Tipo di pratica: certificato di residenza

Numero di copie richieste: 2

=====

COMUNE DI VILLANOVA - RICEVUTA

=====

Richiedente: Laura Greco

Pratica: CERTIFICATO DI RESIDENZA

Copie: 2

Diritti per copia:	16.00
--------------------	-------

Marca da bollo:	16.00
-----------------	-------

TOTALE DA VERSARE:	48.00
--------------------	-------

=====

Il totale è isolato, in fondo, allineato a destra: chi legge lo trova in due secondi senza che voi glielo indichiate.

Cassa NovaStore: lo scontrino

```
ALIQUOTA_IVA = 0.22

articolo = input("Articolo: ").strip().title()
prezzo_grezzo = input("Prezzo unitario in euro (es. 49.90): ")
prezzo = float(prezzo_grezzo.strip().replace(",", "."))
quantita = int(input("Quantita: ").strip())

totale = prezzo * quantita
imponibile = totale / (1 + ALIQUOTA_IVA)
iva = totale - imponibile

print("=" * 50)
print("NOVASTORE - SCONTRINO")
print("=" * 50)

print(f"articolo:<22}{quantita:>4}{prezzo:>10.2f}{totale:>14.2f}")
print("-" * 50)

print(f"Imponibile:':<36}{imponibile:>14.2f}")
print(f"IVA 22%:':<36}{iva:>14.2f}")
print(f"TOTALE:':<36}{totale:>14.2f}")

print("=" * 50)
```

Lo scontrino a schermo

Articolo: cuffie bluetooth

Prezzo unitario in euro (es. 49.90): 49,90

Quantita: 2

```
=====
NOVASTORE - SCONTRINO
=====
Cuffie Bluetooth      2      49.90      99.80
-----
Imponibile:           81.80
IVA 22%:              18.00
TOTALE:               99.80
=====
```

Scorporare moltiplicando per 0.78 è il classico che fa quadrare i conti per difetto.

Cosa hanno in comune

Passo	Comune	Negozio	Ambulatorio
Chiede	richiedente, pratica, copie	articolo, prezzo, quantità	paziente, CF, prestazione, ora
Pulisce	title, upper	title, replace	title, upper, capitalize
Converte	int sulle copie	float e int	niente: sono tutti testi
Calcola	diritti per copie	totale, imponibile, IVA	niente
Presenta	riquadro + totale a destra	tabella a colonne	scheda a etichette

Tre domini, tre record di lunghezza diversa, gli stessi cinque passi. Poliambulatorio Aurora non converte niente: lì il lavoro è tutto di normalizzazione e di forma, e i campi restano di tipo str fino alla stampa.

14. QUELLO CHE OGGI NON SAPPIAMO ANCORA FARE

Il programma non sa dire di no

Cinque input che rompono i programmi di oggi

#	Dove	Si digita	Cosa succede
1	Prezzo	venti	ValueError: could not convert string to float: 'venti'
2	Quantità	(invio a vuoto)	ValueError: invalid literal for int(): ''
3	Quantità	2 pezzi	ValueError: invalid literal for int(): '2 pezzi'
4	Prezzo	1.250,00	ValueError: could not convert ... : '1.250.00'
5	zona;peso;urgenza	SUD,12.5,urgente	ValueError: not enough values (expected 3, got 1)

Il caso 1.250,00: la difesa peggiora le cose

```
prezzo_grezzo = input("Prezzo in euro: ")
prezzo = float(prezzo_grezzo.strip().replace(",", "", "."))
```

--- l'utente digita 1.250,00 (milleduecentocinquanta all'italiana) ---

```
.strip()           ->  "1.250,00"
.replace(",", "", ".") ->  "1.250.00"
float("1.250.00")   ->  due punti: non e' un numero
```

```
ValueError: could not convert string to float: '1.250.00'
```

--- per oggi, la difesa sta nel prompt ---

```
input("Importo in euro, senza separatore delle migliaia (es. 1250.00): ")
```

La normalizzazione preventiva è una difesa, non un'armatura: `.replace()` sostituisce tutte le occorrenze, e qui ha prodotto una stringa con due punti, che non è un letterale numerico valido. Per oggi la difesa sta nel prompt.

E poi c'è quello che non dà errore

Peggior del crash è il caso che passa liscio.

Digitate -40 alla domanda sull'età: `int("-40")` riesce senza protestare, perché il segno meno fa parte del letterale intero.

Il programma stampa "fra dieci anni ne avrai -30".

Nessuna eccezione, un calcolo perfetto, una sciocchezza stampata.

Lo stesso vale per un peso di 0, un IBAN di sei caratteri, una data 99/99/9999.

Il crash almeno urla. Questo no, e finisce in un documento.

La prima linea di difesa: prevenire

Difesa	Come
Prompt che dice il formato	<code>input("Peso in kg, punto decimale (es. 12.5): ")</code>
Prompt che dice cosa non fare	"Importo senza separatore delle migliaia: "
<code>.strip()</code> su tutto	Elimina il problema più frequente in assoluto
<code>.lower()</code> / <code>.upper()</code>	Rende confrontabili le risposte
<code>.replace(",", ".")</code> sui decimali	Copre il caso italiano più comune
Eco dei dati raccolti	Fa vedere l'errore a chi l'ha fatto

Il nome di questo mestiere è validazione preventiva, e per i dati che arrivano da una persona è quella giusta. Difese deboli, e l'ottanta per cento del risultato: gli input sbagliati sono spazi, maiuscole e virgole, non attacchi.

Dove atterra il problema, e cosa viene dopo

Strumento	Che problema risolve
if / elif / else	controllare prima e rifiutare con un messaggio
try / except	sopravvivere dopo, quando il controllo non basta

Il secondo raccoglie l'errore dopo che e' successo, invece di prevenirlo, e non e' materia di queste undici giornate: noi restiamo alla prevenzione.

--- nella prossima giornata lo scontrino di oggi impara tre frasi ---

"Il prezzo deve essere un numero. Hai scritto: venti"

"La quantita deve essere maggiore di zero."

"La zona deve essere NORD, CENTRO, SUD o ISOLE."

Tre messaggi, non tre traceback. Portate il file dell'esercizio 03.2: il 04.4 è 'riparate la cassa di oggi'.

Riepilogo della giornata (1 di 2)

Argomento	Concetti chiave
Ingresso, elaborazione, uscita	I due canali: standard input (stdin) e standard output (stdout)
input()	Sospende l'esecuzione e legge dallo standard input; prompt con lo spazio
Il tipo di ciò che entra	Restituisce sempre un valore di tipo str; l'invio a vuoto dà ""
Il prompt	È un argomento: unità, formato, esempio; le istruzioni lunghe con print()
Igiene del dato	Normalizzazione: .strip() sempre; upper/lower/title; massimo tre in catena
La virgola italiana	.replace(",", ".") prima di float(); non copre 1.250,00
Conversione esplicita	int() e float() su due righe: _grezzo e nome finale; normalizzare prima
ValueError	Eccezione: int("venti"), int(""), int("3.5"); il valore colpevole è fra apici
TypeError	Eccezione da conversione dimenticata: "3" + 1; e il caso muto "3" * 4 -> 3333

Riepilogo della giornata (2 di 2)

Argomento	Concetti chiave
.split() e unpacking	Un record in campi: zona, peso, urgenza = riga.split(";")
Errori di unpacking	not enough / too many; expected = i nomi a sinistra, got 1 = delimitatore
Pulizia dopo lo split	.split() non toglie gli spazi: ogni campo si normalizza per conto suo
Output leggibile	Intestazione, sezioni, "=" * 50, totale isolato in fondo; si chiama eco
Colonne a larghezza fissa	Specificatori: :< testo, :> numeri, :>10.2f euro; la somma fa la riga
Trappole delle colonne	La larghezza è minima: nessuno specificatore taglia, tagliate voi
print() avanzato	I parametri sep ed end e i loro valori predefiniti; [OK], niente emoji
Ciclo di vita del dato	Richiesta, normalizzazione, conversione, calcolo, presentazione
Il limite di oggi	Non sa dire di no: il controllo con if arriva subito; try/except non è materia di queste giornate

Grazie.

shaping the skills of tomorrow

